

Assignment 4

Markers, Computer Vision and Sensor Fusion (20P)

Goals

You are familiar with basic functions of the OpenCV library. You can write programs to detect fiducial markers, for example Aruco. You know about perspective transformation and can apply it to extract and wrap regions of an image. You can implement a simple sensor fusion.

1. Perspective Transformation (5P)

Recommended Packages: OpenCV

Write a program called *image_extractor.py* that loads and displays an image with OpenCV. By clicking into the displayed image, users should be able to select four points. The selected region is then extracted and perspectively warped to a rectangle. After all four points have been selected, the warped result should be displayed. By pressing *ESC*, users should be able to discard the changes and start over. By pressing *S* in the result view, the image should be saved.

Paths to the input file and output destination, as well as the result's resolution should be specified via command line parameters.

Score

- (1P) The image is successfully loaded and displayed
- (1P) Selecting the corner points works and there is visual feedback for the user
- (2P) Perspective transformation to the target resolution works
- (1P) Command line parameters and shortcuts work

2. AR Game (10P)

Recommended Packages: OpenCV, numpy, pygame

Create a program called *AR_game.py*. The program should read out your webcam image. Use a board with an Aruco marker in each corner, extract the region between the markers, and transform it to a rectangle with the resolution of your webcam. Keep in mind that not all webcams have the same resolution! Display the extracted and warped rectangle in a pygame application. Now, add some game mechanics based on the image in the extracted rectangle. The game mechanics must include tracking an object (not enhanced through Aruco markers). For example, players could be able to use their finger to destroy targets or to move things around.

Score

- (2P) the region of interest is detected, extracted, transformed, and displayed
- (4P) Objects (such as fingers) in the region of interest are tracked reliably and interaction with game objects works
- (2P) Game mechanics work

(1P) Performance and latency is ok

(1P) The program does not crash

3. Sensor Fusion (5P)

Recommended Packages: OpenCV, numpy, pygame, DIPPID

Create a program called *sensor_fusion.py*. The program should read your webcam image and track a board with Aruco markers the same way as in exercise 2 (extract the region between the markers and transform it to a rectangle). Attach another Aruco marker (ID 5) to the back of your smartphone and track its position within the board rectangle. The program should start a pygame application that displays the position of that Aruco marker as red dot. (Note: you can also use the DIPPID application to display an Aruco marker (ID 23) if that's easier for you).

Additionally, read accelerometer data from your mobile device using DIPPID. Implement a complementary filter that fuses the camera position with the accelerometer data to produce position prediction (it may work better if you multiply a fixed scalar number to the accelerometer). Continuously display the predicted position as green dot in your pygame application. The arrow keys should be usable to adjust the weight alpha at runtime. Pressing Button 1 in the DIPPID app should reset the prediction. Try your code with different alpha values and reflect on what you notice.

Score

(1P) The moving marker is tracked, and its position is displayed in the pygame window

(1P) Accelerometer data is received from DIPPID and integrated over time

(1P) The complementary filter blends camera and accelerometer data and the prediction is displayed in the pygame window

(1P) The filter weight alpha can be adjusted at runtime, and the prediction can be reset using DIPPID

(1P) Write a short paragraph reflecting on the implementation and how different weights affect the prediction

Code Quality: bad code quality and missing documentation can lead to loss of points.